

Function	Register Usage	Description
Clrscr	-	Clears the console window and locates the cursor at the top left corner.
Crlf	-	Writes the end-of-line sequence to the console window.
WriteBin	EAX	Writes an unsigned 32-bit integer to the console in ASCII binary format.
WriteChar	AL	Writes a single character to the console window.
WriteDec	EAX	Writes an unsigned 32-bit integer to the console in decimal format.
WriteHex	EAX	Writes a 32-bit integer to the console in hexadecimal format.
WriteInt	EAX	Writes a signed 32-bit integer to the console in decimal format.
WriteString	EDX (OFFSET String)	Writes a null-terminated string to the console.
ReadChar	AL	Waits for a single character to be typed and returns it in AL.
ReadDec	EAX	Reads an unsigned 32-bit integer from the keyboard.
ReadHex	EAX	Reads a 32-bit hexadecimal integer from the keyboard, terminated by the enter key.
ReadInt	EAX	Reads a signed 32-bit integer from the keyboard, terminated by the enter key.
ReadString	EDX (OFFSET), ECX (SIZE)	Reads a string from the keyboard, terminated by the enter key.
ReadKey	AL	Waits for a single key press and stores its ASCII code in AL.

Delay	EAX	Pauses execution for a specified interval (in milliseconds).
Randomize	-	Seeds the random number generator with a unique value.
Random	EAX	Generates a random number based on the state of the random number generator (returns the random value in EAX).
RandomRange	ECX (Max), EAX (Result)	Generates a random number in the range 0 to Max-1 (Max provided in ECX). Returns result in EAX.
Random32	EAX	Generates a 32-bit pseudorandom integer in the range 0 to FFFFFFFFh. Returns the random value in EAX.
DumpRegs	-	Displays values of registers (EAX, EBX, ECX, EDX, ESI, EDI, ESP, EIP, EFLAGS).
DumpMem	ESI (Starting OFFSET), ECX (LengthOf), EBX (Type)	Writes a block of memory to the console in hexadecimal format.
GetDateTime	Assuming the function returns: ; EAX = year ; EBX = month ; ECX = day ; EDX = hour ; ESI = minute ; EDI = second	Gets the current date and time from the system.
GetMaxXY	DX (col), AX (row)	Gets the number of columns and rows in the console window buffer.
GetTextColor	AL	Returns the active foreground and background text colors in the console window.
Gotoxy	DH (row), DL (col)	Locates the cursor at a specific row and column in the console.
GetMilliseconds	EAX	Returns the number of milliseconds elapsed since midnight in EAX.

MsgBox	EDX (OFFSET String), EBX (OFFSET Title)	Displays a pop-up message box with the message at the address in EDX and title at the address in EBX.
MsgBoxAsk	EDX (OFFSET String), EBX (OFFSET Title)	Displays a yes/no question in a pop-up message box. (EAX=6 for YES, EAX=7 for NO)
SetTextColor	EAX (Foreground + (Background * 16))	Sets the foreground and background colors for subsequent text output.
WaitMsg	-	Displays a message and waits for the Enter key to be pressed.
isDigit	AL (input character)	Checks if the character in AL is a digit (0-9).
ParseDecimal32	EDX (offset of string), EAX (result)	Converts an unsigned integer string to binary, storing the result in EAX.
ParseInteger32	EDX (offset of string), EAX (result)	Converts a signed integer string to binary, storing the result in EAX.
Str_compare	EDX (offset of first string), EBX (offset of second string) ; Assuming the result is returned in EAX ; EAX == 0 if strings are equal ; EAX < 0 if string1 < string2 ; EAX > 0 if string1 > string2	Compares two null-terminated strings, returning a result based on lexicographical order.
Str_copy	EDX (offset of destination), EBX (offset of source)	Copies a source string to a destination string, both null-terminated.
StrLength	EDX (offset of string), EAX (length)	Returns the length of a null-terminated string, with the result in EAX.
Str_trim	EDX (offset of string) AL (char to remove)	Removes unwanted characters (such as spaces or specified characters) from a string.

Value Description**0Dh** Carriage Return (CR) - Moves the cursor to the beginning of the line.**0Ah** Line Feed (LF) - Moves the cursor down to the next line.**0** Null Terminator - Indicates the end of the string for functions like WriteString.**Color Codes for SetTextColor**

Color Name	Foreground (Lower 4 bits)	Background (Upper 4 bits)	Combined Value (EAX)
Black	0	0	0x00
Blue	1	0	0x01
Green	2	0	0x02
Cyan	3	0	0x03
Red	4	0	0x04
Magenta	5	0	0x05
Brown	6	0	0x06
Light Gray	7	0	0x07
Dark Gray	8	0	0x08
Light Blue	9	0	0x09
Light Green	10	0	0x0A
Light Cyan	11	0	0x0B
Light Red	12	0	0x0C
Light Magenta	13	0	0x0D
Yellow	14	0	0x0E
White	15	0	0x0F

Operation Name	Flags Cleared	Flags Set According to Value / Always
AND	OF, CF	ZF (Zero Flag) - Set if the result is zero SF (Sign Flag) - Set if the result is negative PF (Parity Flag) - Set according to the parity of the result
OR	OF, CF	ZF (Zero Flag) - Set if the result is zero SF (Sign Flag) - Set if the result is negative PF (Parity Flag) - Set according to the parity of the result
NOT	None	None - Inverts bits but does not affect flags
XOR	OF, CF	ZF (Zero Flag) - Set if the result is zero SF (Sign Flag) - Set if the result is negative PF (Parity Flag) - Set according to the parity of the result
TEST (Non Destructive)	OF, CF	ZF (Zero Flag) - Set if the result is zero SF (Sign Flag) - Set if the result is negative PF (Parity Flag) - Set according to the parity of the result
CMP (Non Destructive)	None	ZF (Zero Flag) - Set if operands are equal SF (Sign Flag) - Set if the result is negative OF (Overflow Flag) - Set if there is an overflow in signed subtraction CF (Carry Flag) - Set if the destination < source
STC	None	CF (Carry Flag) - Set to 1 (always sets the carry flag)
CLC	CF	CF (Carry Flag) - Cleared to 0 (always clears the carry flag)
BT	None	tests a specific bit in the destination operand and copies that bit into the Carry Flag (CF) CF (Carry Flag) - Set if the tested bit is 1; cleared if the tested bit is 0 16-bit, 32-bit only

Unsigned	ZF	CF
destination < source	0	1
destination > source	0	0
destination = source	1	0

Signed	FLAGS
destination < source	SF != OF
destination > source	SF == OF
destination = source	ZF = 1

Instruction	Condition	Flags Checked	Description
Unconditional Jumps			
JMP	Always	None	Jump to a specified address unconditionally.
Conditional Jumps (Based on Specific Flags)			
JO	Overflow	OF = 1	Jump if overflow flag is set.
JNO	No Overflow	OF = 0	Jump if overflow flag is cleared.
JC	Carry	CF = 1	Jump if carry flag is set.
JNC	No Carry	CF = 0	Jump if carry flag is cleared.
JS	Sign	SF = 1	Jump if sign flag is set (negative).
JNS	No Sign	SF = 0	Jump if sign flag is cleared (non-negative).
JP / JPE	Parity Even	PF = 1	Jump if parity flag is set (even parity).

JNP / JPO	Parity Odd	PF = 0	Jump if parity flag is cleared (odd parity).
Jumps Based on Equality / Comparison			
JE / JZ	Equal / Zero	ZF = 1	Jump if zero flag is set (equal).
JNE / JNZ	Not Equal / Not Zero	ZF = 0	Jump if zero flag is cleared (not equal).
Jumps Based on Unsigned Comparison			
JA / JNBE	Above (unsigned)	CF = 0 and ZF = 0	Jump if first operand > second (unsigned).
JAE / JNB	Above or Equal (unsigned)	CF = 0	Jump if first operand ≥ second (unsigned).
JB / JNAE	Below (unsigned)	CF = 1	Jump if first operand < second (unsigned).
JBE / JNA	Below or Equal (unsigned)	CF = 1 or ZF = 1	Jump if first operand ≤ second (unsigned).
Jumps Based on Signed Comparison			
JG / JNLE	Greater (signed)	ZF = 0 and SF = OF	Jump if first operand > second (signed).
JGE / JNL	Greater or Equal (signed)	SF = OF	Jump if first operand ≥ second (signed).
JL / JNGE	Less (signed)	SF ≠ OF	Jump if first operand < second (signed).
JLE / JNG	Less or Equal (signed)	ZF = 1 or SF ≠ OF	Jump if first operand ≤ second (signed).
Loop-Based Jumps			
LOOP	Counter not zero	ECX/CX ≠ 0	Decrement ECX and jump if it is not zero.
LOOPE / LOOPZ	Counter not zero and Equal	ECX/CX ≠ 0, ZF = 1	Decrement ECX and jump if it is not zero and ZF is set.
LOOPNE / LOOPNZ	Counter not zero and Not Equal	ECX/CX ≠ 0, ZF = 0	Decrement ECX and jump if it is not zero and ZF is cleared.

Instruction	Operation (Multiply/Divide)	Flags Affected	Operand Sizes	Description
SHL (Shift Left Logical)	Multiplies by 2 for each shift	CF (Carry Flag) - set to last shifted-out bit OF (Overflow Flag) - set if original MSB changes	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Shifts bits left, filling in 0s on the right. The sign bit is not preserved, and each shift moves bits left, causing a 2x multiplication per shift.
SHR (Shift Right Logical)	Divides by 2 for each shift	CF - set to last shifted-out bit OF - unaffected	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Shifts bits right, filling in 0s on the left. This operation does not maintain the sign bit, making it suited for unsigned division by 2 per shift.
SAL (Shift Arithmetic Left)	Multiplies by 2 for each shift	CF - set to last shifted-out bit OF - set if original MSB changes	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Identical to SHL in behavior. It shifts bits left, filling in 0s on the right and discarding the sign bit. This is typically used for signed multiplication by 2 per shift.
SAR (Shift Arithmetic Right)	Divides by 2 for each shift	CF - set to last shifted-out bit OF - unaffected	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Shifts bits right, maintaining the original sign bit (MSB) and filling in the MSB as it shifts. Used for signed division by 2 per shift.
ROL (Rotate Left)	None (rotation)	CF - set to the last rotated-out bit OF - set if MSB changes	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Rotates all bits to the left, with the leftmost bit wrapping around to the rightmost position. Sign bit is not preserved, and there is no zero fill.
ROR (Rotate Right)	None (rotation)	CF - set to the last rotated-out bit OF - set if MSB changes	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Rotates all bits to the right, with the rightmost bit wrapping around to the leftmost position. The sign bit is not preserved, and there is no zero fill.
RCL (Rotate through Carry Left)	None (rotation)	CF - set to the last rotated-out bit OF - set if MSB changes	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Similar to ROL, but includes the carry flag as part of the rotation. The sign bit is not preserved, and the carry flag plays an active role in the rotation sequence.

RCR (Rotate through Carry Right)	None (rotation)	CF - set to the last rotated-out bit OF - set if MSB changes	Value - 8-bit, 16-bit, 32-bit (reg/mem) Shift - 8-bit (CL/imm)	Similar to ROR, but includes the carry flag in the rotation. This rotates bits right, with the carry flag taking the leftmost position. The sign bit is not preserved.
SHLD (Double-Precision Shift Left)	Multiplies by 2 for each shift	CF - set to last shifted-out bit OF - not affected	16/32-bit destination, 16/32-bit source Shift: 8-bit (CL/imm)	Shifts the destination operand left by a specified count, filling opened bit positions on the right with the most significant bits of the source operand.
SHRD (Double-Precision Shift Right)	Divides by 2 for each shift	CF - set to last shifted-out bit OF - not affected	16/32-bit destination, 16/32-bit source Shift: 8-bit (CL/imm)	Shifts the destination operand right by a specified count, filling opened bit positions on the left with the least significant bits of the source operand.

Explanation Summary:

- Logical Shifts (SHL, SHR): Used for unsigned numbers, filling the new positions with zeros. SHL multiplies, while SHR divides.
- Arithmetic Shifts (SAL, SAR): Used for signed numbers, with SAL acting like SHL and SAR preserving the sign bit.
- Rotates (ROL, ROR, RCL, RCR): Move bits circularly without filling with zeros. RCL and RCR involve the carry flag.
- Double-Precision Shifts (SHLD, SHRD): Shifts across two operands, filling bits from a second source into the primary target.

Operation	Description	Operand Size	Registers Used	Flags Affected																
MUL	Unsigned multiply; multiplies AL, AX, or EAX by an operand.	8-bit, 16-bit, 32-bit	<table><tr><th>Multiplicand</th><th>Multiplier</th><th>Product</th></tr><tr><td>AL</td><td>reg/mem8</td><td>AX</td></tr><tr><td>AX</td><td>reg/mem16</td><td>DX:AX</td></tr><tr><td>EAX</td><td>reg/mem32</td><td>EDX:EAX</td></tr></table>	Multiplicand	Multiplier	Product	AL	reg/mem8	AX	AX	reg/mem16	DX:AX	EAX	reg/mem32	EDX:EAX	CF, OF set if result doesn't fit in lower half				
Multiplicand	Multiplier	Product																		
AL	reg/mem8	AX																		
AX	reg/mem16	DX:AX																		
EAX	reg/mem32	EDX:EAX																		
IMUL	Signed multiply; multiplies AL, AX, or EAX by an operand. Supports immediate, 1-operand, and 2-operand forms.	8-bit, 16-bit, 32-bit dest = 2 × mul1 size, mul2 can be the same size as mul1 or smaller	<table><tr><th>Multiplicand</th><th>Multiplier</th><th>Product</th></tr><tr><td>AL</td><td>reg/mem8</td><td>AX</td></tr><tr><td>AX</td><td>reg/mem16</td><td>DX:AX</td></tr><tr><td>EAX</td><td>reg/mem32</td><td>EDX:EAX</td></tr></table>	Multiplicand	Multiplier	Product	AL	reg/mem8	AX	AX	reg/mem16	DX:AX	EAX	reg/mem32	EDX:EAX	CF, OF set if result exceeds destination size				
Multiplicand	Multiplier	Product																		
AL	reg/mem8	AX																		
AX	reg/mem16	DX:AX																		
EAX	reg/mem32	EDX:EAX																		
DIV	Unsigned division; divides accumulator by operand.	8-bit, 16-bit, 32-bit	<table><tr><th>Dividend</th><th>Divisor</th><th>Quotient</th><th>Remainder</th></tr><tr><td>AX</td><td>reg/mem8</td><td>AL</td><td>AH</td></tr><tr><td>DX:AX</td><td>reg/mem16</td><td>AX</td><td>DX</td></tr><tr><td>EDX:EAX</td><td>reg/mem32</td><td>EAX</td><td>EDX</td></tr></table>	Dividend	Divisor	Quotient	Remainder	AX	reg/mem8	AL	AH	DX:AX	reg/mem16	AX	DX	EDX:EAX	reg/mem32	EAX	EDX	-
Dividend	Divisor	Quotient	Remainder																	
AX	reg/mem8	AL	AH																	
DX:AX	reg/mem16	AX	DX																	
EDX:EAX	reg/mem32	EAX	EDX																	

IDIV	Signed division; divides accumulator by operand, adjusting for sign.	8-bit, 16-bit, 32-bit	<table><tr><th>Dividend</th><th>Divisor</th><th>Quotient</th><th>Remainder</th></tr><tr><td>AX</td><td>reg/mem8</td><td>AL</td><td>AH</td></tr><tr><td>DX:AX</td><td>reg/mem16</td><td>AX</td><td>DX</td></tr><tr><td>EDX:EAX</td><td>reg/mem32</td><td>EAX</td><td>EDX</td></tr></table>	Dividend	Divisor	Quotient	Remainder	AX	reg/mem8	AL	AH	DX:AX	reg/mem16	AX	DX	EDX:EAX	reg/mem32	EAX	EDX	-
Dividend	Divisor	Quotient	Remainder																	
AX	reg/mem8	AL	AH																	
DX:AX	reg/mem16	AX	DX																	
EDX:EAX	reg/mem32	EAX	EDX																	

Operand Sign Extension Instructions

Instruction	Operation	Registers Used	Description
CBW	Byte to Word Conversion	AL to AX	Converts the signed byte in AL to a signed word in AX by sign-extending AL.
CWD	Word to Doubleword Conversion	AX to DX:AX	Converts the signed word in AX to a signed doubleword in DX:AX by sign-extending AX.
CDQ	Doubleword to Quadword Conversion	EAX to EDX:EAX	Converts the signed doubleword in EAX to a signed quadword in EDX:EAX by sign-extending EAX.

Additional Instructions for Arithmetic Operations

Operation	Description	Operand Size	Registers Used	Flags Affected
ADC	Adds operands (same size) with the carry flag (for chained additions).	8-bit, 16-bit, 32-bit	General-purpose	CF, OF, ZF, SF, PF, AF
SBB	Subtracts operands (same size) with the borrow flag (CF), useful for chained subtractions.	8-bit, 16-bit, 32-bit	General-purpose	CF, OF, ZF, SF, PF, AF

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

```
.data
; 8-bit signed
smallest8BitSigned  BYTE -128      ; Smallest: -128 (0x80)
largest8BitSigned   BYTE 127       ; Largest: 127 (0x7F)

; 8-bit unsigned
smallest8BitUnsigned BYTE 0        ; Smallest: 0
largest8BitUnsigned  BYTE 255      ; Largest: 255 (0xFF)

; 16-bit signed
smallest16BitSigned  WORD -32768   ; Smallest: -32768 (0x8000)
largest16BitSigned   WORD 32767    ; Largest: 32767 (0x7FFF)

; 16-bit unsigned
smallest16BitUnsigned WORD 0       ; Smallest: 0
largest16BitUnsigned WORD 65535    ; Largest: 65535 (0xFFFF)

; 32-bit signed
```

```
smallest32BitSigned  DWORD -2147483648 ; Smallest: -2147483648 (0x80000000)
largest32BitSigned   DWORD 2147483647  ; Largest: 2147483647 (0x7FFFFFFF)
```

```
; 32-bit unsigned
```

```
smallest32BitUnsigned DWORD 0          ; Smallest: 0
largest32BitUnsigned  DWORD 4294967295 ; Largest: 4294967295 (0xFFFFFFFF)
```